Experiment No -10 (Group D)

Title:-

Graph Traversal of City Routes Using BFS and DFS

Problem statement:-

Consider a particular area in your city. Note the popular locations A, B, C . . . in that area. Assume these locations represent nodes of a graph. If there is a route between two locations, it is represented as connections between nodes.

Find out the sequence in which you will visit these locations, starting from (say A) using (i) BFS and (ii) DFS. Represent a given graph using an adjacency matrix to perform DFS and an adjacency list to perform BFS

Pre-requisite:-

- **Fundamental concepts of Data Structures and Algorithms.**
- **Graph terminology such as vertex (node), edge, path, and adjacency.**
- **Representation of graphs using adjacency matrix and adjacency list.**
- **Basic programming concepts such as arrays, lists, functions, and loops.**
- **Basic understanding of stack and queue concepts.**
- **Familiarity with a programming language such as Python.**

Objective:-

- To understand the representation of a real-world area as a **graph** using locations as vertices and routes as edges.
- To represent the given graph using an **adjacency matrix**.
- To represent the given graph using an **adjacency list**.
- To implement **Depth First Search (DFS)** using an adjacency matrix.
- To implement **Breadth First Search (BFS)** using an adjacency list.
- To determine the sequence of locations visited starting from a given location **A**.
- To understand the difference between **BFS and DFS traversal techniques**.

Software and Hardware requirements:-

Software :-

Operating System recommended:- 64-bit Open source Linux or Windows or macOS

Programming tools recommended: - Python 3.x

Programming IDE/Editor: IDLE, VS Code, PyCharm, or any Python-supported editor

Hardware :-

Processor: Intel Core i3 or equivalent and above

RAM: Minimum 2 GB

Storage: Minimum 100 MB free space

Keyboard and Monitor: Required for entering input and viewing output.

Outcome:-

After completing this practical, students will be able to:

- Identify and model real-world locations and routes as a **graph**.
- Construct an **adjacency matrix** for a given graph.
- Construct an **adjacency list** for a given graph.
- Perform **DFS traversal** using an adjacency matrix.
- Perform **BFS traversal** using an adjacency list.
- Determine and compare the visiting sequence produced by **BFS and DFS**.
- Apply graph traversal techniques to real-world problems such as **route finding, network traversal, and connectivity analysis**.

Theory:-

A **graph** is a non-linear data structure used to represent relationships between different objects.

A graph consists of:

- **Vertices (Nodes):** Represent objects or locations.
- **Edges:** Represent connections or routes between the locations.

For example, consider four popular locations:

A = Railway Station, B = Bus Stand, C = Hospital, D = College

If there is a direct route between two locations, we represent it by an **edge**.

For this practical, we use: A, B, C, D as the nodes of the graph.

The connections are:

A — B

| |

C — D

Graph Representation

There are different ways to represent a graph. In this practical, we use two methods.

1. Adjacency Matrix

An adjacency matrix uses a **2-dimensional array (matrix)** to represent connections between nodes.

An adjacency matrix is a square matrix of size **N × N**, where N is the number of vertices.

For our graph:

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

Meaning of values

- 1 → There is a connection between two locations.
- 0 → There is no connection.

For example:

A → B = 1

means there is a route between A and B.

Similarly:

A → D = 0

means there is no direct route between A and D.

Because this is an **undirected graph**, the matrix is symmetric.

2. Adjacency List

An adjacency list stores the **list of neighboring nodes** for every node.

In an adjacency list, we store the neighboring locations of each node.

For the given graph:

```
adj_list = {
   'A': ['B', 'C'],
   'B': ['A', 'D'],
   'C': ['A', 'D'],
   'D': ['B', 'C']
}
```

This means:

A → B, C

B → A, D

C → A, D

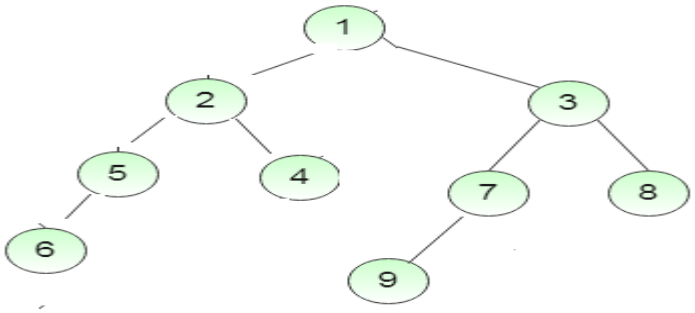
D → B, C

For example: A: B, C means A is directly connected to B and C.

Breadth First Search

It is an algorithm for traversing or searching tree or graph data structure. In BFS one starts at root and explore all the child of every nodes of every parent node before backtracking.

In short we try to traverse tree level wise before completing previous level we can't go to next level.



BFS traversal of given tree can be

1 2 3 5 4 7 8 6 9 OR 1 3 2 8 7 4 5 9 6

Steps followed for BFS traversal

1. pick root node as starting node

2.Insert root nod on the queue

3. Delete the front node from the queue & visit it.

4. if child nodes are present for visited node then insert them on queue.

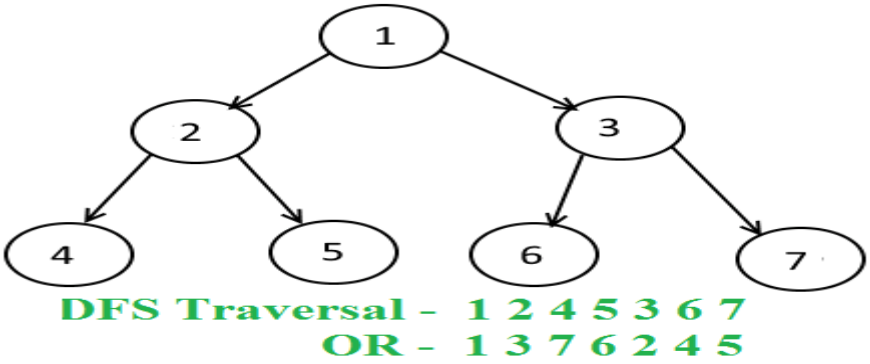
5.Repat step 3 and 4 till queue is not empty

Depth First Search

It is an algorithm for traversing or searching tree or graph data structure. In DFS one starts at the root and explore as far as possible along each branch before backtracking.

Here we are trying to traverse depth-wise (i.e. come down traversing from root to leaf node in the same branch and the backtrack.)

DFS traversal of given tree can be



For DFS traversal stack data structure is used for backtracking.

Steps followed for DFS traversal of tree

1) Pick root node as a starting node

2)Push root node on the stack

3) Pop node from the stack and visit it (print)

4) Push left child and right of visited node on stack (if not NULL)

5) Repeat step 3 and 4 till stack is not empty

Conclusion:-

The practical demonstrates how a real-world area can be represented using a **graph**, where locations are represented as nodes and routes are represented as edges.

DFS was performed using an adjacency matrix, while **BFS** wasa performed using an adjacency list. The traversal sequences starting from location **A** were obtained successfully.

This practical helps in understanding graph representation and the practical applications of **BFS and DFS traversal techniques**.

Viva Questions:-

1. What is a graph?.
2. What is BFS?
3. What is DFS?
4. Which data structure is used in BFS? .
5. Which data structure is used in DFS?
6. What is an adjacency matrix?.
7. What is an adjacency list?
8. Why is visited used?
9. What is the main difference between BFS and DFS?
10. What does 1 represent in an adjacency matrix
11. What is the advantage of an adjacency matrix?